.

Prova di Algoritmi e Strutture Dati

Non si possono consultare né appunti né testi

20 Maggio 2025

Il punteggio è espresso su 36 punti, e farà media aritmetica con il primo esonero.

Esercizio 1 **[punti 8]** Dato un albero binario di ricerca T, trovare il secondo minimo in T, ossia il nodo la cui chiave è l'elemento di rango due nelle chaivi di T ordinate in senso crescente.

- Spiegare a parole la soluzione proposta
- Dare lo pseudo codice dell'algoritmo
- Discutere la complessità dell'algoritmo proposto.

Il secondo minimo è il minimo del figlio di destra del nodo che contiene il minimo o il padre del nodo che contiene il minimo. Infatti il padre del minimo è sempre più grande del minimo stesso.

Pseudocodice: Assumiamo che ogni nodo contenga un attributo puntatore al nodo padre.

```
1 Predecessore_Minimo(Nodo x)
2 while x.left ≠ NULL do
3     x = x.left // Cerchiamo il massimo
4 if x.right ≠ NULL then
5     x = x.right
6     while x.left ≠ NULL do
7         x = x.left
      return: x
8 else
      return: x.p
```

Esercizio 2 **[punti 8]** Dato un grafo $G = (V, E)$ non orientato e connesso, visitare G e restituire il tipo di ogni arco in E.

- Dare lo pseudo codice dell'algoritmo;
- Discutere la complessità dell'algoritmo proposto;
- Quali possono essere i tipi degli archi in G?

Pseudocodice: Visitiamo il grafo con DFS e supponiamo che il grafo $G = (V, E)$ sia memorizzato con Vettore di Adiacenza. Quest'ultima scelta è solo per abituarci a sfruttare questa struttura dati. Assumiamo i vertici numerati da 1 a $|V|$. Il grafo è non orientato e connesso. Quindi non è necessario invocare più volte la DFS-visit. G ha una sola componente connessa e gli archi sono in T o in B.

```
1  DFS(Grafo G)
2  for ogni u ∈ G.V do
3  |   u.colore = WHITE
4  |   u.π = NULL
5  DfsVisit(G, 1)
```

```
1  DfsVisit(Grafo G, Vertice u)
2  u.colore = GRAY
3  for [G.V[u]] ≤ i ≤ [G.V[u + 1] − 1] do
4  |   v = G.E[i]
5  |   if v.colore == WHITE then
6  |   |   v.π = u
7  |   |   G.E[i].type = "T" //
8  |   |   DfsVisit(G, v)
9  |   else
10 |   |   if v.colore == GRAY and v ≠ u.π then
11 |   |   |   G.E[i].type = "B"
12 u.colore = BLACK
```

Nota. Si noti che **non** si può risolvere l'esercizio invocando la DFS del grafo orientato. Infatti usando la DFS dei grafi orientati, per ogni arco del grafo non orientato, sarà dato il tipo a due archi orientati. Si avrà così che un arco $e = (u, v)$ 'backward' di G (con $v \neq \pi(u)$) sarà battezzato come arco in B quando visitato nella direzione (u, v) ma come arco 'Forward' quando visitato nella direzione (v, u). Quindi alcuni archi saranno 'battezzati' due volte con valori diversi e questo è errato perchè si tratta di un solo arco nel grafo non-orientato. Per risolvere questo problema bisogna evitare di battezzare gli archi che raggiungono nodi neri. In altre parole, dobbiamo seguire la procedura che abbiamo studiato per i grafi non orientati!

Esercizio 3 **[punti 10]** Sia $G = (V, E)$ un grafo orientato, pesato $w : E \to \mathbb{R}^+$, dato un vertice $d \in V$, e dato un intero $k \geq 1$, trovare i cammini di costo minimo da ogni vertice in V verso la destinazione d con al più k archi.

- Spiegare a parole la tecnica risolutiva;
- Dare lo pseudo codice dell'algoritmo;
- Discutere la complessità dell'algoritmo proposto.

Per risolvere l'esercizio, eseguiamo il metodo tropicale moltiplicando k la matrice che rappresenta G con il vettore corrispondente alla colonna d della matrice G. W è la matrice di adiacenza con pesi che rappresenta G.

Pseudocodice:

```
1 Extended-Shortest-Paths(Matrix W[n × n], Matrix H[n × 1])
2 for i = 1 to n do
3 |   for k = 1 to n do
4 |   |   if W[i,k] + H[k] < H[i] then
5 |   |   |   H[i] = W[i,k] + H[k]
6 |   |   // scelta del 'successore'
  return: H
```

```
1 Soluzione(Matrix W[n × n], Matrix R[n × 1]) //
2 Array R^1[n] = W[∗, d] //
3       R^1[n] costo archi entranti in d, ossia la d-esima colonna di W
4 for m = 2 to k do
5 |   R^m = Extended-Shortest-Paths(W,R^{m−1}) // R^m[i] è il cammino di costo minimo
  |     verso il nodo d che parte da i e attraversa al più m archi.
  return: R^m
```

Il numero massimo di archi k da attraversare non ha nulla a che vedere con il costo dei cammini minimi.
Considerate il grafo $G = (V, E)$ con $V = \{u, 1, 2, 3, v\}$ ed $E = \{(u, 1; 2); (u, 2; 1); (1, v; 7); (2, 3; 5); (3, v; 1)\}$ dove la notazione $(r, s; z)$ indica l'arco (r, s) con peso $w(r, s) = z$. Il cammino minimo con $k \leq 2$ è il cammino $(u, 1, v)$ con peso 9. L'algoritmo di Dijkstra non lo genera e neppure lo trova come cammino ammissibile. Provate per convincervi.

Esercizio 4 **[punti 10]** Sia $G = (V, E)$ un grafo non orientato, connesso e pesato, tale che non vi siano due archi con lo stesso peso. Sia M l' albero di copertura di peso minimo in G. Dire quali delle seguenti affermazioni sono vere e spiegare:

- M è unico
- L'arco di costo minimo appartiene a M
- L'arco di costo massimo non appartiene a M
- Se si aggiunge un arco in G, il nuovo albero di copertura M' ha lo stesso numero di archi come M
- Se si aggiunge un vertice in G e un insieme di archi per connettere il nuovo vertice a G, il nuovo albero di copertura M' ha lo stesso numero di archi come M
- Se si aggiunge un vertice in G e un insieme di archi per connettere il nuovo vertice a G, tutti gli archi in M appartengono anche al nuovo albero di copertura

Risposte

- M è unico perchè gli archi hanno costi tutti distinti - vero
- L'arco di costo minimo appartiene a M -vero. Considerate la partizione che separa i vertici di questo arco e applicate la regola blue.
- L'arco di costo massimo non appartiene a M -falso. Considerate la partizione che separa i vertici di questo arco e considerate il taglio. Un arco diverso potrebbe essere quello da inserire nella soluzione. Se l'arco di costo massimo è l'unico nel taglio, allora esso entra nella soluzione.
- Se si aggiunge un arco in G, il nuovo albero di copertura M' ha lo stesso numero di archi come M -vero perchè il numero di archi dipende dal numero di vertici in G.
- Se si aggiunge un vertice in G e un insieme di archi per connettere il nuovo vertice a G, il nuovo albero di copertura M' ha lo stesso numero di archi come M -falso perchè il numero di archi dipende dal numero di vertici e il numero di vertici del nuovo grafo è aumentato.
- Se si aggiunge un vertice in G e un insieme di archi per connettere il nuovo vertice a G, tutti gli archi in M appartengono anche al nuovo albero di copertura -falso. Dopo aver connesso il nuovo vertice al grafo. per ogni arco inserito nel grafo si deve verificare se l'arco nel ciclo che forma ha costo massimo o meno.